Mukesh Patel School of Technology Management & Engineering (Mumbai Campus)

IT DEPARTMENT

Project Synopsis

Title of Project:

Sentinel: AI-Assisted Smart Home IoT Digital Forensics and Incident Response Platform

Team Members:

Atharva Rangnekar - K053 | Aarya Rao - K054 | Ayra Jha - K028

Under the supervision of

Prof. Rohit Suryawanshi

Table of Contents

1. Title of the Project

Sentinel: AI-Assisted Smart Home IoT Digital Forensics and Incident Response Platform

2. Team Members

1. Atharva Rangnekar (K053)
2. Aarya Rao (K054)
3. Ayra Jha (K028)

Project Guide: Prof. Rohit Suryawanshi

3. Introduction

Smart homes combine cameras, door sensors, voice assistants, smart plugs, thermostats and gateways that produce network records, telemetry, authentication traces and state changes. During an incident, relevant evidence is distributed across devices and expressed through different schemas, clocks and identifiers. A single alert rarely provides a defensible account; the investigator must normalize records, relate devices and reconstruct a chronology.

Sentinel is an industry-oriented academic prototype for this problem. It accepts public IoT datasets and simulated smart-home incidents, normalizes heterogeneous logs, detects suspicious activity using rules and behavioural baselines, assigns explainable risk scores, correlates events, reconstructs timelines and presents findings in a web dashboard. It does not replace an enterprise SIEM, an IoT security platform or a certified forensic suite.

3.1. Problem Statement

Public IoT datasets are useful for repeatable experiments, but their fields and labels differ and they do not directly provide a case-oriented forensic view. A compact system is required that retains evidence provenance while producing deterministic filtering, sorting, scoring, correlation, timeline and graph outputs. Generative AI creates a second risk: if it calculates or invents values, the investigation becomes irreproducible. Sentinel therefore confines the local language model to evidence-grounded narration and validated layout planning; Python remains the only computational authority.

4. Literature Survey

4.1. IoT DFIR and Public Datasets

NIST describes digital forensics as a capability that supports incident response through collection, examination, analysis and reporting [1]. In IoT environments, heterogeneity, limited storage, volatile evidence and clock inconsistency make these activities difficult [4]. A sound prototype must preserve the raw-record link, distinguish observed time from import time, document normalization decisions and avoid treating correlation as proof of causation.

TON_IoT contains heterogeneous IoT/IIoT telemetry, operating-system traces and network traffic captured in a cyber-range environment [2]. CICIoT2023 provides a large IoT attack benchmark with benign and malicious traffic from numerous devices and attack scenarios [3]. Sentinel uses these datasets to test adapters, rules and performance, and uses small simulated cases for exact correlation and timeline tests. These sources support academic evaluation but do not establish production readiness or legal admissibility.

Rule-based logic is retained because each alert can show its condition and contributing records. Behavioural baselines add deviations in rates, peers, protocols or active periods. Risk is a bounded, versioned combination of stored factors such as severity, confidence, repetition, device criticality and cross-device corroboration. NetworkX represents device/entity relationships, while stable sorting and explicit correlation windows produce reproducible graphs and timelines. NIST log-management guidance supports structured, maintained security-log processes [5].

4.2. Comparison with Existing Platforms

Platform	Enterprise Strength	Sentinel Position
Microsoft Defender for IoT	IoT/OT discovery, network sensors, behavioural analytics, vulnerability information and Microsoft ecosystem integration [8].	Offline public-dataset DFIR prototype with transparent derivation and smart-home scenarios.
AWS IoT Device Defender	AWS IoT configuration audits, device behaviour metrics and mitigation actions [9].	Cloud-independent evidence analysis; no fleet enforcement or autonomous mitigation.
Splunk Enterprise Security	Broad ingestion, correlation searches, risk-oriented triage and incident workflows [10].	Purpose-built canonical IoT event model and deterministic educational pipeline.
IBM QRadar	Event/flow normalization, correlation rules and prioritized offenses [11].	Smaller case reconstruction, evidence timelines, device graphs and bounded local AI.

These products are mature enterprise systems with deployment, scale, content and integration capabilities outside Sentinel's scope. The comparison is therefore contextual rather than competitive: Sentinel demonstrates selected DFIR concepts transparently using academic data.

4.3. Research Gap

A gap exists between intrusion-detection experiments and complete, inspectable DFIR workflows. Many prototypes classify rows but do not preserve a traceable path through normalization, risk factors, cross-device links, timeline membership and visual values. Adaptive dashboards may also allow generative systems to produce unchecked content. Sentinel addresses this gap by combining deterministic analytics, evidence-grounded local AI and a visualization contract that permits layout selection without permitting invented numerical values or arbitrary frontend code.

5. Why is this particular topic chosen?

Smart-home IoT was selected because it is familiar yet forensically demanding. Several vendors and device types interact in one private environment, so evidence is fragmented and privacy-sensitive. Public datasets permit repeatable experiments without collecting household telemetry, while simulated incidents create controlled multi-device narratives. The domain also provides a practical way to study explainability, correlation, timeline reconstruction, privacy-conscious local AI and safe adaptive visualization.

6. Objective and Scope of the project

6.1. Objectives and Scope

- Ingest selected TON_IoT, CICIoT2023 and simulated evidence with source hashes, adapter versions and validation errors.
- Normalize records into a canonical event model while retaining raw-record locators.
- Use deterministic Python for filtering, stable sorting, rules, baselines, risk scoring, correlation, timelines, chart aggregates and graph values.
- Provide a React dashboard containing metric/severity cards, timelines, line/bar/pie charts, evidence tables, device graphs and recommendation panels.
- Use local Qwen 9B Q4_K_M through Ollama only for summaries, grounded explanations, investigation chat, email drafting and visualization layout planning.
- Evaluate correctness, reproducibility, detection/correlation quality, visualization safety, usability and performance.

In scope are batch imports, case management, deterministic analysis, dashboard review, local AI assistance, SQLite storage, SMTP alerts and report export. Out of scope are production-device acquisition, firmware extraction, live interception, autonomous containment, legal-admissibility claims and replacement of enterprise products. The study uses public datasets and simulated incidents, not production smart-home devices.

6.2. Architecture and Workflow

The layered architecture separates dataset adapters, the canonical event model, deterministic analytics, persistence, API services, the visualization registry and the AI assistant. Imported evidence and model output are both treated as untrusted. FastAPI and Pydantic validate requests and responses; SQLite stores cases and derived artifacts through a repository boundary that can later support PostgreSQL.

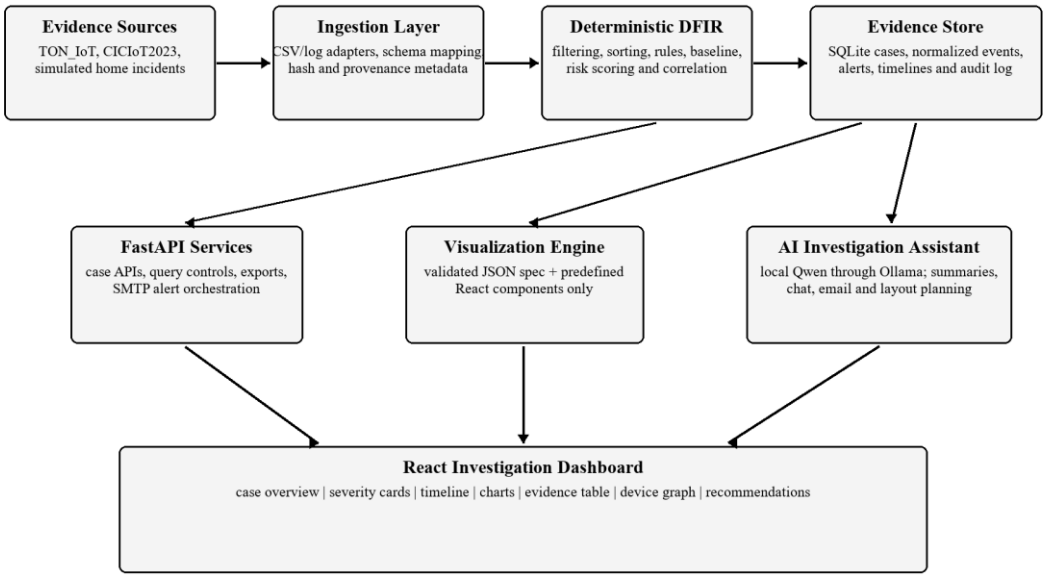

Figure 1. Proposed Sentinel system architecture

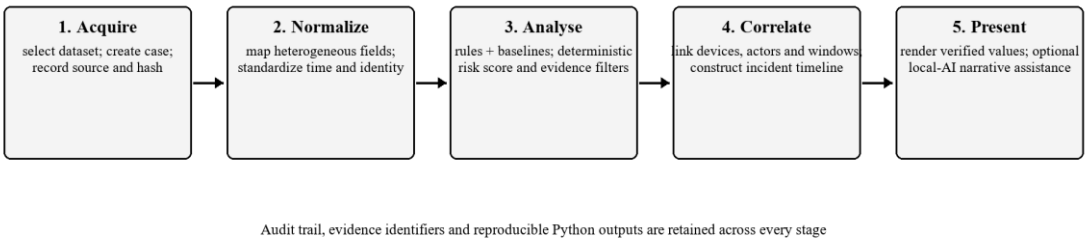

Figure 2. Evidence-to-investigation workflow

The workflow registers and hashes evidence, validates and normalizes rows, performs rules/baselines and factorized scoring, correlates events by declared entity keys and time windows, and renders verified results. Every derived artifact records evidence identifiers and configuration versions. The model is invoked only after deterministic analysis and its failure does not disable core DFIR functions.

6.3. Module Descriptions

Module	Responsibility
Evidence and Adapter Layer	Registers cases and hashes; validates dataset columns; emits normalized events or documented rejections.
Detection and Risk Engine	Executes rules and baselines; returns evidence lists, condition traces, factor breakdowns and bounded scores.
Correlation and Timeline Engine	Links events using configured keys/windows; records link reasons; stable-sorts case chronology.

Module	Responsibility
Graph and Query Engine	Uses NetworkX and Pandas for graph measures, filters, aggregates and chart-ready values.
API, Storage and Alerts	FastAPI/Pydantic services, SQLite repositories, audit history, report export and investigator-approved SMTP.
Visualization and AI	Renders allow-listed components and provides bounded, evidence-grounded local assistance.

6.4. Visualization Engine

The Visualization Engine maps a Pydantic-validated JSON specification to predefined React components implemented with Recharts and React Flow. The specification contains a schema version, allow-listed component name, deterministic data reference, existing-field encodings and bounded layout metadata. It cannot contain JavaScript, JSX, SQL, Python, free-form expressions or numerical arrays invented by the model. Unknown components, fields or case references are rejected and a safe default layout is used.

6.5. AI Investigation Assistant

Qwen 9B Q4_K_M runs locally through Ollama. Structured-output support allows schema-constrained JSON [7], with Pydantic validation and evidence-identifier checks added by Sentinel. The assistant may summarize deterministic findings, explain a rule trace or correlation path, answer questions from selected evidence, draft an email for human approval and plan component placement. It cannot create alerts, calculate scores, send mail autonomously, execute code, modify evidence or present an unsupported inference as fact. Prompts and responses are logged with model, template version and evidence identifiers. The literature recognizes both opportunities and reliability/privacy challenges for LLMs in cybersecurity [6].

6.6. Testing and Evaluation

Area	Method and Measure
Normalization	Boundary and malformed fixtures; exact normalized values and explicit rejection reasons.
Reproducibility	Repeated golden runs; exact equality of order, scores, correlations, timelines and graph data.
Detection	Labeled dataset subsets; precision, recall, F1-score and confusion matrix, with class imbalance reported.
Correlation	Curated multi-device cases; incident grouping, link precision/recall and ordered-event agreement.
Visualization/AI	Unknown components, prompt injection, absent evidence and malformed JSON; unsafe output rejected and citations resolved.
Integration/Performance	API, persistence, SMTP mock and user tasks; latency, throughput and memory reported with test conditions.

Acceptance emphasizes correctness and traceability rather than an exaggerated accuracy claim. Dataset results will be reported as prototype measurements under declared samples, hardware and configuration. A disabled or unavailable Ollama service must leave ingestion, analysis, correlation, timelines, graphs and manual layouts usable.

6.7. Security, Limitations, Future Scope and Conclusion

Evidence is processed locally, uploads are size/type checked, paths are normalized, secrets are excluded from model context and case actions are audited. The interface distinguishes deterministic findings from AI narrative and avoids attributing identity or intent solely from an anomaly, IP address or correlation score.

The main limitation is reliance on public testbeds and simulations, which cannot reproduce every device, proprietary protocol, clock error, vendor cloud or household context. Future work includes opt-in laboratory collectors, MQTT and gateway sources, clock-drift estimation, richer entity resolution, PostgreSQL, role-based access control, immutable evidence storage, SIEM/SOAR export and controlled

user studies. Sentinel concludes as a privacy-conscious academic demonstration of explainable smart-home DFIR: Python calculates every forensic and visual value, while local AI improves readability only within a bounded, validated role.

7. Hardware and Software Requirements

7.1. Hardware

- 64-bit quad-core processor; 16 GB RAM minimum (24-32 GB recommended for local inference).
- 30 GB free SSD space; optional supported GPU; stable local network connection.

The minimum configuration supports development, deterministic dataset analysis and CPU-based model execution on representative samples. Additional memory is recommended because Pandas may hold normalized records and derived tables in memory while Ollama loads the quantized model. A supported GPU improves response time but is not required for correctness; all core forensic modules remain usable when the model is disabled. SSD storage is preferred for dataset imports, SQLite case files, model weights and report exports.

7.2. Software

- Frontend: React, Vite, Tailwind CSS, shadcn/ui, Recharts and React Flow.
- Backend: Python, FastAPI, Pandas, NetworkX and Pydantic.
- Database and AI: SQLite, optional future PostgreSQL, Ollama with Qwen 9B Q4_K_M.
- Alerts and development: SMTP, Git, GitHub, Codex, Pytest and a modern browser.

The frontend is responsible only for presentation and validated user interaction. FastAPI exposes case, evidence, analysis, visualization and assistant endpoints through Pydantic request/response contracts. Pandas performs schema transformation and aggregation, while NetworkX builds the correlation graph and returns calculated nodes, edges and metrics. SQLite is sufficient for a single-user academic prototype; the repository boundary permits future PostgreSQL migration without changing forensic algorithms.

Ollama runs locally and is called through a backend service that supplies bounded evidence context. SMTP credentials are stored outside the source repository and email is sent only after explicit investigator approval. Git and GitHub maintain the project history, automated tests protect deterministic behaviour, and Codex may assist implementation and review without becoming part of the runtime evidence pipeline.

8. Final Deliverables

- Documented React/FastAPI source repository and reproducible setup instructions.
- Dataset adapters, normalized-event schema and representative smart-home cases.
- Deterministic Python detection, scoring, correlation, timeline, chart and graph modules.
- Dashboard, validated Visualization Engine, local AI assistant, SQLite case history and SMTP support.
- Automated tests, evaluation results, exports and final report documenting limitations and future work.

Deliverable Area	Acceptance Evidence
Evidence pipeline	Successful import logs, documented field mappings, hashes and canonical-schema validation.
Deterministic analytics	Golden-case matches for rules, risk factors, graph edges, chart values and ordered timelines.
Visualization and AI	Allow-listed JSON schema tests, invalid-spec rejection and proof that AI cannot modify computed values.
Reporting and privacy	Auditable exports, investigator-approved email flow, secret handling and local case-data cleanup.

The submitted implementation package will include sample configuration, database initialization, supported dataset-field mappings and representative simulated cases. Each analytical result will expose the evidence identifiers and configuration version needed to reproduce it. The dashboard will demonstrate a complete investigation path from import and normalization to alerts, risk factors, cross-device links, timeline review and export.

Delivery acceptance will require successful automated tests, schema validation for every API and visualization response, a clean installation walkthrough and repeatable golden-case results. The final report will record dataset selections, test conditions, measured results and known limitations. Source files, documentation and demonstration data will exclude passwords, SMTP secrets, private household telemetry and model-generated claims that are not supported by evidence.

A demonstration case will accompany the submission to make assessment repeatable. It will contain a controlled sequence of simulated events, the expected normalized records, rule matches, risk-factor breakdown, correlation edges and ordered timeline. Reviewers will therefore be able to compare the dashboard and exported report with fixed expected outputs, inspect provenance links back to source rows and confirm that disabling the language model does not alter any forensic value. A short operator guide will identify supported inputs, configuration boundaries, privacy safeguards and the procedure for clearing local case data after evaluation.

The evaluation bundle will also preserve the configuration used for each run: adapter version, rule-set version, baseline window, scoring weights, correlation window, dataset subset and software environment. Where performance figures are reported, the synopsis implementation will distinguish measured import, analysis, query and rendering times from qualitative observations about local-model responsiveness. Screenshots and exports will use pseudonymous device identifiers. Any discrepancy between an expected and observed golden-case result will be treated as a defect to investigate rather than adjusted through an AI-generated explanation. This traceability makes the prototype suitable for academic review while keeping its claims proportionate to public datasets and simulated incidents.

9. References

[1] K. Kent, S. Chevalier, T. Grance, and H. Dang, Guide to Integrating Forensic Techniques into Incident Response, NIST SP 800-86, 2006, doi: 10.6028/NIST.SP.800-86.

[2] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems," IEEE Access, vol. 8, pp. 165130-165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[3] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," Sensors, vol. 23, no. 13, Art. 5941, 2023, doi: 10.3390/s23135941.

[4] A. O. Akinbi, "Digital Forensics Challenges and Readiness for 6G Internet of Things (IoT) Networks," WIREs Forensic Science, vol. 5, no. 6, e1496, 2023, doi: 10.1002/wfs2.1496.

[5] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92, 2006, doi: 10.6028/NIST.SP.800-92.

[6] H. Xu et al., "Large Language Models for Cyber Security: A Systematic Literature Review," CoRR, abs/2405.04760, 2024, doi: 10.48550/arXiv.2405.04760.

[7] Ollama, "Structured Outputs," Ollama Documentation. [Online]. Available: https://docs.ollama.com/capabilities/structured-outputs. Accessed: Jul. 14, 2026.

[8] Microsoft, "Defender for IoT - OT Architecture and Components," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/azure/defender-for-iot/organizations/architecture. Accessed: Jul. 14, 2026.

[9] Amazon Web Services, "What is AWS IoT Device Defender?" AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/iot-device-defender/latest/devguide/what-is-device-defender.html. Accessed: Jul. 14, 2026.

[10] Splunk, "Correlation Search Overview for Splunk Enterprise Security," Splunk Documentation. [Online]. Available: https://help.splunk.com/en/splunk-enterprise-security-7/administer/7.3/correlation-searches/correlation-search-overview-for-splunk-enterprise-security. Accessed: Jul. 14, 2026.

[11] IBM, "Custom Rules in IBM QRadar," IBM Documentation. [Online]. Available: https://www.ibm.com/docs/en/qradar-on-cloud?topic=siem-custom-rules. Accessed: Jul. 14, 2026.